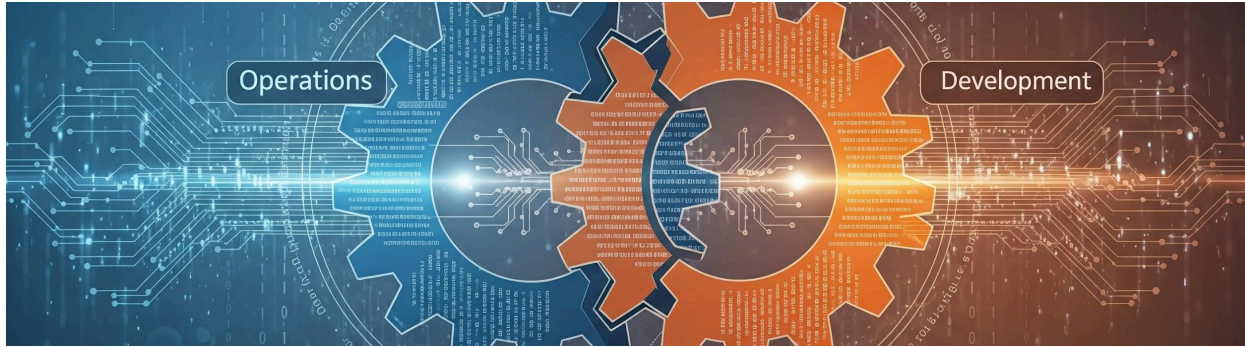




Site Reliability Engineering (SRE)

1. Why SRE?

Today's applications should **always work**, even when users increase and systems grow.

Earlier, operations teams worked manually:

- Restart servers
- Fix issues after failure
- Work in silos

This caused:

- Frequent downtime
- Slow recovery
- Human errors
- Customer dissatisfaction

SRE is needed because:

- Modern systems are **complex and distributed**
- Downtime directly causes **business and money loss**
- Manual work does not scale
- Companies need **fast releases without breaking systems**

SRE helps keep systems stable while allowing fast development.

2. What is SRE?

Site Reliability Engineering (SRE) means using **software engineering techniques to manage IT operations**.

Instead of fixing problems manually, SREs:

- Write code
- Build automation
- Design reliable systems

Simple definition:

SRE = Engineers who keep applications reliable using automation, monitoring, and code.

SRE was introduced by **Google** to manage large-scale systems like Search, Gmail, and YouTube.

3. What SRE Can Do

SRE teams are responsible for **production systems**.

Main responsibilities:

- **Monitoring & Alerting** – Track system health (CPU, memory, latency, errors)
- **Incident Management** – Handle production issues quickly
- **Root Cause Analysis (RCA)** – Find why the issue happened
- **Automation** – Replace manual work with scripts and tools
- **Performance Optimization** – Reduce latency and improve speed
- **Capacity Planning** – Ensure systems can handle future traffic
- **High Availability** – Design systems with backup and failover

👉 Goal: **Less downtime, faster recovery, more reliability**

4. DevOps vs SRE

While SRE and DevOps share the common goal of improving system reliability and accelerating delivery, SRE can be viewed as an *implementation* of the DevOps philosophy.

Here is a comparison of their focuses:

Aspect	DevOps	SRE
Nature	Culture & mindset	Engineering discipline
Primary Goal	Fast delivery	Reliable delivery
Focus Area	CI/CD, collaboration	Availability, uptime, performance
Responsibility	Shared by Dev & Ops	Dedicated SRE team
Metrics	Deployment frequency, lead time	SLOs, SLIs, error budgets
Failure Handling	Fix and move fast	Learn, improve reliability

5. SRE Principles / Golden Rules

1. Reliability Is a Feature

Reliability is treated like any other product feature (login, payment, search, etc.).

- If an app is fast but frequently down, users will stop using it
- Business trust depends on system stability
- Reliability must be planned, designed, tested, and improved

Example:

An e-commerce site during a sale must stay available. Even if no new features are added, improving uptime is valuable.

2. Service Level Objectives (SLOs)

SLOs define **how reliable a service should be**.

- They are measurable targets
- Based on user experience
- Help teams make decisions

Common SLO examples:

- 99.9% uptime per month
- 95% of requests respond within 200 ms
- Error rate below 1%

Why SLOs matter:

- Prevent over-engineering
- Set clear expectations
- Help balance speed vs stability

3. Error Budget

Error Budget is the **allowed amount of failure** based on SLOs.

- If SLO is 99.9%, allowed downtime is ~43 minutes/month
- Teams can use this budget for experiments and releases

Rules:

- If error budget is available → release faster
- If error budget is exhausted → focus on stability

Benefit: Encourages innovation **without risking reliability**.

4. Eliminate Toil

Toil is:

- Manual

- Repetitive
- Reactive
- Low-value work

Examples of toil:

- Manually restarting servers
- Fixing the same alert daily
- Copy-paste deployments

SRE goal:

- Automate toil using scripts, tools, and platforms
- Free time for engineering work

5. Monitoring Over Alerting

Monitoring shows **what is happening** in the system.

Good monitoring focuses on:

- User experience
- Latency
- Errors
- Traffic
- Saturation

Alerting rules:

- Alert only when action is required
- Avoid noisy alerts
- Alert on symptoms, not causes

Example:

Alert when users can't place orders, not when CPU spikes slightly.

6. Blameless Postmortems

After an incident, teams write a **postmortem** to learn.

Key points:

- No blaming individuals
- Focus on system weaknesses
- Document timeline and root cause

- Create action items to prevent repeat issues

Benefit:

- Encourages honesty
- Improves systems
- Builds a healthy culture

7. Automation First

Automation is core to SRE.

Principle:

If a task is done more than once, automate it.

Automate:

- Deployments
- Scaling
- Recovery
- Testing
- Monitoring

Result:

- Fewer human errors
- Faster recovery
- Consistent operations